

5-Mark Questions & Answers — Data Structures Module 1

Topic 1: Concept of ADT

Q1. Explain ADT in detail. How is an ADT implemented in a programming language? Give an example of Stack ADT with its operations.

1. Detailed Concept of ADT

An **Abstract Data Type (ADT)** is a high-level mathematical specification of a data structure. It defines:

- **Data Objects:** The type and organization of data elements.
- **Operations:** The set of functions/operations that can be performed, along with input parameters and expected behaviors.
- **Implementation Independence:** ADT specifies **what** operations are performed, leaving **how** they are implemented to the programmer.

2. Implementation in Programming Languages

In C/C++, ADTs are implemented using:

1. **Header Files (.h):** Declare structure types and function prototypes (the public interface).
2. **Source Files (.c):** Define private structure details and concrete logic.
3. **Encapsulation / Classes (C++/Java):** Use **class** with **private** data members and **public** member functions.

3. Stack ADT Example & Operations

A **Stack ADT** follows Last-In-First-Out (LIFO) behavior:

```
// Interface Definition (Stack ADT)
typedef struct Stack {
    int items[100];
    int top;
} Stack;

// Core Operations:
void push(Stack *s, int val) {
    if (s->top == 99) return; // Overflow
    s->items[++(s->top)] = val;
}

int pop(Stack *s) {
    if (s->top == -1) return -1; // Underflow
    return s->items[(s->top)--];
}

int peek(Stack *s) {
    return (s->top != -1) ? s->items[s->top] : -1;
}
```

Q2. Describe how ADT provides abstraction and information hiding with a real-life example (e.g., List ADT or Queue ADT).

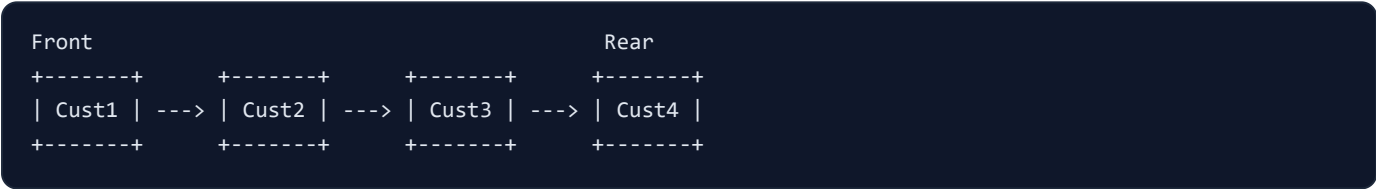
1. Abstraction and Information Hiding

- **Abstraction** simplifies complex reality by modeling classes appropriate to the problem, exposing only essential operational features.
- **Information Hiding** prevents client code from directly modifying internal variables, requiring all interactions through defined interface functions.

2. Real-Life Example: Queue ADT (Ticket Counter Analogy)

Think of a **Ticket Booking System (Queue ADT)**:

- **Public Interface:**
- `enqueue(customer)` : Join the line at the rear.
- `dequeue()` : Serve the customer at the front.
- **Hidden Implementation:** Customers don't know whether the line uses a contiguous array or linked list nodes behind the digital display.

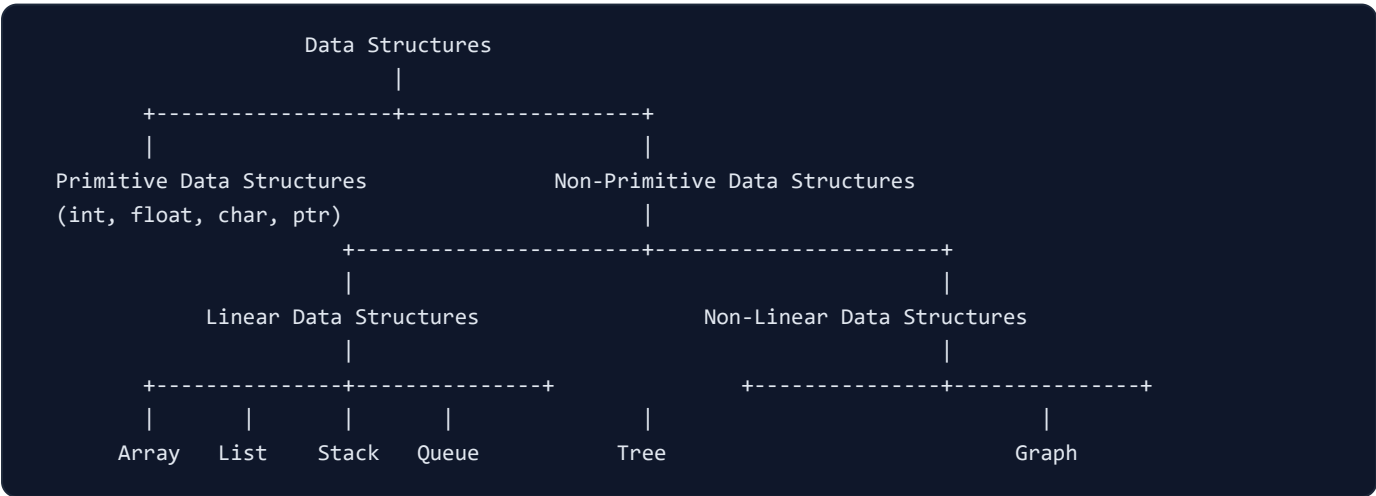


- **Benefits:**
- Software engineers can switch the queue backend from array to linked list for better dynamic memory scaling without breaking any existing client code.

Topic 2: Types of Data Structures

Q3. Explain the classification of data structures in detail with examples and diagrams.

Data structures are categorized into two primary hierarchies:



1. Primitive Data Structures

Base data types directly operated on by CPU instructions:

- `int` (Integer values, e.g., `42`)
- `float` / `double` (Floating point numbers, e.g., `3.14`)
- `char` (Single character bytes, e.g., `'A'`)
- `pointer` (Memory addresses, e.g., `int *ptr`)

2. Non-Primitive Data Structures

Complex user-defined structures that group primitive data types:

- **Linear Data Structures:** Elements form a single sequential line where each element has a unique predecessor and successor (except endpoints).
 - *Array*: Homogeneous contiguous memory.
 - *Linked List*: Nodes linked via pointers.
 - *Stack*: LIFO structure.
 - *Queue*: FIFO structure.
- **Non-Linear Data Structures:** Elements form hierarchical or network connections.
 - *Tree*: Hierarchical parent-child relationship (e.g., File system, BST).
 - *Graph*: Many-to-many vertex-edge network (e.g., Social networks, Google Maps routing).

Q4. Discuss the various types of data structures along with their real-world applications.

Data Structure	Type	Key Characteristics	Real-World Application
Array	Linear, Static	O(1) random access, contiguous memory.	Storing image pixels, tabular data.
Linked List	Linear, Dynamic	O(1) insertion/deletion without memory shifting.	Undo functionality in text editors, Music playlist.
Stack	Linear, LIFO	Push/Pop at top only.	Function call stack, Browser back button, Expression parsing.
Queue	Linear, FIFO	Enqueue at rear, Dequeue at front.	CPU process scheduling, Printer job queues.
Binary Tree / BST	Non-Linear	Hierarchical parent-child nodes.	Database indexing (B-Trees), File systems.
Graph	Non-Linear	Vertices connected by edges.	Social network friend connections, GPS Navigation (Shortest path).

Topic 3: Operations on Data Structures

Q5. Explain in detail the various operations performed on data structures: Traversing, Searching, Insertion, Deletion, Sorting, and Merging, with suitable examples for each.

1. Traversing

Accessing and processing every element in a structure exactly once.

- *Example*: Printing array elements: `for(i=0; i<n; i++) print(arr[i]);`

2. Searching

Finding the position of a given target key.

- Example: Linear Search in `arr = {10, 25, 40}` for target `25` returns index `1`.

3. Insertion

Adding a new element at a specific index or position.

- Example: Inserting `15` at index `1` in `{10, 20}` shifts `20` right to produce `{10, 15, 20}`.

4. Deletion

Removing an existing element and re-organizing memory.

- Example: Deleting `15` from `{10, 15, 20}` shifts `20` left to produce `{10, 20}`.

5. Sorting

Arranging elements in ascending or descending sequence.

- Example: Sorting `{9, 2, 7}` produces `{2, 7, 9}` using Bubble/Quick Sort.

6. Merging

Combining two sorted structures into one unified sorted structure.

- Example: Merging `{1, 4}` and `{2, 5}` yields `{1, 2, 4, 5}`.

Q6. Discuss time complexity considerations for basic operations on data structures.

Time complexity measures how algorithm execution time grows as input size **n** increases:

Data Structure	Access	Search	Insertion	Deletion
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Singly Linked List	$O(n)$	$O(n)$	$O(1)$ (at head)	$O(1)$ (at head)
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Binary Search Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

- **Trade-off Analysis:** Arrays offer instant $O(1)$ random access by index but suffer slow $O(n)$ insertions. Linked Lists sacrifice random access ($O(n)$) to achieve fast $O(1)$ dynamic insertion and deletion.

Topic 4: Arrays & Multi-Dimensional Arrays

Q7. Explain in detail how a 2-D array is represented in memory using row-major and column-major order. Derive the address calculation formula for both.

A 2-D array **A[M][N]** with lower bounds **LBR, LBC**, Base Address **B**, and element size **w**:

1. Row-Major Order (RMO)

Elements are stored row by row sequentially.

- **Derivation:** To locate $A[i][j]$, skip $(i - \text{LBR})$ rows (each with N elements) plus $(j - \text{LBC})$ columns.

Equation:

$$\text{Address}(A[i][j]) = B + w \times [(i - \text{LBR}) \times N + (j - \text{LBC})]$$

2. Column-Major Order (CMO)

Elements are stored column by column sequentially.

- **Derivation:** To locate $A[i][j]$, skip $(j - \text{LBC})$ columns (each with M elements) plus $(i - \text{LBR})$ rows.

Equation:

$$\text{Address}(A[i][j]) = B + w \times [(j - \text{LBC}) \times M + (i - \text{LBR})]$$

Q8. What is a sparse matrix? Explain its representation using triplet/array representation with an example.

1. Definition

A **Sparse Matrix** is a matrix in which the number of zero elements is significantly higher than the number of non-zero elements.

2. Triplet Representation

Instead of storing all $M \times N$ entries, store non-zero values in a $K \times 3$ array where row 0 holds matrix dimensions:

Example Matrix (4×4):

Equation:

$$\begin{bmatrix} 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 \\ 0 & 0 & 0 & 9 \end{bmatrix}$$

Triplet Table:

Row	Column	Value	Note
----	----	----	----
4	4	3	(Total Rows, Total Cols, Non-Zero Count)
0	2	3	$A[0][2] = 3$
2	1	5	$A[2][1] = 5$
3	3	9	$A[3][3] = 9$

Q9. Explain multi-dimensional arrays with memory representation and address calculation with a numerical example.

1. Numerical Problem

Given a 2-D array $A[10... 20][30... 50]$ stored in **Row-Major Order** with:

- Base Address $B = 2000$
- Element size $w = 4$ bytes

Find the address of $A[15][40]$.

2. Given Data

- $LBR = 10, UBR = 20 \Rightarrow M = 20 - 10 + 1 = 11$ rows
- $LBC = 30, UBC = 50 \Rightarrow N = 50 - 30 + 1 = 21$ columns
- Target indices: $i = 15, j = 40$

3. Calculation

Equation:

$$\text{Address}(A[15][40]) = B + w \times [(i - LBR) \times N + (j - LBC)]$$

Equation:

$$\text{Address}(A[15][40]) = 2000 + 4 \times [(15 - 10) \times 21 + (40 - 30)]$$

Equation:

$$\text{Address}(A[15][40]) = 2000 + 4 \times [5 \times 21 + 10]$$

Equation:

$$\text{Address}(A[15][40]) = 2000 + 4 \times [105 + 10] = 2000 + 4 \times 115 = 2000 + 460 = 2460$$

Answer: Memory Address of $A[15][40] = 2460$.

Topic 5: String Manipulation

Q10. Explain various string manipulation operations (concatenation, comparison, substring, reverse) with algorithms/examples.

1. **Concatenation:** Appends string S_2 to end of S_1 .
 - *Algorithm:* Find $'\backslash 0'$ in S_1 , copy characters of S_2 sequentially, append $'\backslash 0'$.
2. **Comparison:** Compares S_1 and S_2 ASCII values.
 - *Algorithm:* Compare $S_1[i]$ vs $S_2[i]$. If difference $\neq 0$, return difference. If both reach $'\backslash 0'$, return 0 .
3. **Substring:** Extracts k characters starting at index pos .
 - *Algorithm:* Loop from $i = pos$ to $pos + k - 1$ and copy into result buffer.
4. **Reverse:** Reverses character order in-place.
 - *Algorithm:* Swap $S[left]$ and $S[right]$ while incrementing $left$ and decrementing $right$.

Q11. Write and explain an algorithm for pattern matching (Naive/Brute-force method) with an example.

1. Naive Pattern Matching Algorithm

```
Algorithm NaivePatternSearch(Text T[0..n-1], Pattern P[0..m-1]):
1. For i = 0 to (n - m):
2.     j = 0
3.     While (j < m AND T[i + j] == P[j]):
4.         j = j + 1
5.     If j == m:
6.         Print "Pattern found at index " + i
```

2. Numerical Example

- **Text T:** "A A B A C A" (n = 6)
- **Pattern P:** "A B A" (m = 3)
- **Execution:**
- **i=0:** Compare T[0..2] "A A B" vs "A B A" ** ⇒ ** Mismatch at j=1.
- **i=1:** Compare T[1..3] "A B A" vs "A B A" ** ⇒ ** Match at j=3!
- **Result:** Pattern found at index 1 .

Q12. Discuss string storage representations (fixed length vs linked list representation) in detail.

Feature	Fixed-Length Array Representation	Variable Linked List Representation
Storage Structure	Contiguous static char array <code>char s[100];</code> .	Dynamic list of nodes containing <code>char data</code> and <code>Node *next</code> .
Memory Efficiency	Wastes unused array space if string is short.	Allocates exact memory needed per character (plus pointer overhead).
Insertion / Deletion	Requires O(n) character shifting.	O(1) pointer updating without shifting.
Access Speed	O(1) fast indexed access <code>s[i]</code> .	O(n) traversal to reach index <code>i</code> .

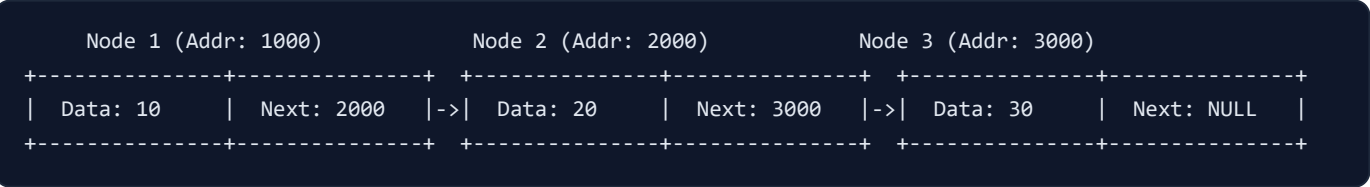
Topic 6: Self-Referential Structures with Pointers

Q13. Explain self-referential structures in detail with syntax, memory representation, and a diagram showing how nodes are linked.

A **Self-Referential Structure** is a C structure that contains at least one pointer member pointing to a structure of the same type:

```
struct Node {
    int data;           // Data Payload
    struct Node *next; // Self-referential pointer
};
```

Memory Layout Diagram:



- Node 1 at address 1000 stores value 10 and pointer 2000 .
- Node 2 at address 2000 stores value 20 and pointer 3000 .
- Node 3 at address 3000 stores value 30 and pointer NULL .

Q14. Write a program/algorithm to create and traverse a simple linked list using self-referential structures.

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void traverse(struct Node *head) {
    struct Node *temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    // Dynamic allocation of 3 nodes
    struct Node *head = (struct Node*)malloc(sizeof(struct Node));
    struct Node *second = (struct Node*)malloc(sizeof(struct Node));
    struct Node *third = (struct Node*)malloc(sizeof(struct Node));

    head->data = 100;
    head->next = second;

    second->data = 200;
    second->next = third;

    third->data = 300;
    third->next = NULL;

    traverse(head);
    return 0;
}

```

Q15. Discuss the advantages of self-referential structures over arrays for dynamic data handling.

1. **Dynamic Memory Allocation:** Memory is allocated at runtime as needed via `malloc()`, eliminating static array capacity limits and memory wastage.
2. **Efficient Insertion & Deletion:** Inserting or deleting a node requires only updating pointer addresses (**O(1)** time complexity), whereas arrays require shifting up to **n** elements (**O(n)** time complexity).
3. **No Contiguous Memory Requirement:** Nodes can be located anywhere in heap memory fragments, whereas arrays require a single contiguous block of memory.